

TRABALHO PRÁTICO

A ADOSMELHORES é uma empresa de formação e pretende uma aplicação para gestão dos seus ativos.

Crie a classe abstrata *Funcionário* e crie por herança as classes *diretor*, *secretaria*, *formador* e *coordenador*. As classes *diretor*, *secretaria*, *formador* e *coordenador* derivadas da classe *Funcionário*.

- Na classe *Diretor*, deve ser incluído isenção de horário, bónus mensal, e carro da empresa (sim/não).
- Na classe *secretaria*, deve estar mencionado a qual *diretor* reporta e a área.
- Na classe *Formador*, deve ter mencionado qual área leciona, se a disponibilidade é pós-laboral, laboral ou ambas e valor hora.
- Na classe *coordenador* deve existir uma lista de formadores associados.
- O *funcionário* deve ter ID, nome, morada, contacto, data de fim de contrato e data de registo criminal.
- Na classe *funcionário*, deve ser acrescentado todos os atributos relevantes e genéricos a todas as subclasses.

GRUPO I

1. Crie uma classe *Empresa* que possua uma lista de *funcionários*.
2. Criar um Form que permita as seguintes opções:
 - a. Inserir novo *funcionário* (qualquer tipo)
 - b. Alterar registo criminal (atualizado / expirado).
 - c. Ver *funcionários* com contrato valido para a data atual, de sistema.
 - d. Ver *funcionários* com registo criminal expirado.
 - e. Calcular o valor a pagar um *funcionário* formador, indicando data início e final da formação, contabilizar 6 horas por dia.
 - f. Exportar a informação sobre os *funcionários* para um ficheiro CSV.
3. Criar um sistema de alocação de formadores.(academia de formação)
4. Calcular o valor total de despesa por mês da empresa.
5. Acrescente um simulador de data, ou seja, na primeira execução da aplicação deve aparecer a data de sistema mas acrescente uma opção para avançar um dia. Sempre que esta opção é selecionada devem ser verificados todos os *funcionários* e mostrar alarme caso tenha sido atingida a data de término de um contrato ou registo criminal.

Nota: O sistema deve ser projetado e programado em C#.

Entregáveis:

- Projeto_nomes_dos_formandos.zip
- Relatório_nomes_dos_formandos.docx
- Link para o GIT. (adicionar user: rcanelas)

Critérios de avaliação:

Individual: 50%

- **Domínio e Conhecimento Técnico (20%):**

Capacidade de explicar o "como" e não apenas o "o quê". Justificação das escolhas lógicas diretamente no código.

- **Oralidade (10%):**

Capacidade de transmitir o conhecimento inerente ao projeto sem conceitos complexos e sem ser vago ou ambíguo.

- **Utilização do Git e Contribuição Individual (15%):**

Frequência e Clareza dos Commits: Avaliação do histórico de commits. As mensagens devem ser explicativas e o trabalho deve ser submetido de forma faseada (não apenas um "mega-commit" final). Rastreabilidade: O código presente no projeto final deve ser condizente com as submissões feitas pelo formando no repositório.

- **Autoavaliação (5%):**

Reflexão crítica sobre o próprio desempenho e aprendizagem.

Coletiva: 50%

- **Qualidade do Código e Boas Práticas (10%):**

Indentação, comentários pertinentes e ausência de código redundante.

- **Complexidade e Funcionalidade (10%):**

Cumprimento dos requisitos técnicos propostos para o projeto.

- **Relatório Técnico e Documentação (15%):**

Estrutura: Introdução, arquitetura da solução, dificuldades encontradas e conclusões.

Conteúdo: Demonstração visual (screenshots/diagramas) e explicação das principais funcionalidades.

- **Gestão de Projeto via Git (5%):**

Organização das branches (se aplicável), resolução de conflitos e equilíbrio de trabalho entre os membros (visível no histórico do Git).

- **Originalidade e Criatividade (5%):**

Interface, funcionalidades extra ou abordagens inovadoras ao problema.

- **Cumprimento de Prazos e Soft Skills (5%):**

Entrega pontual de todos os artefactos e capacidade de comunicação oral durante a apresentação.